

this project currently shows the calendar in the front end, but it only shows the recurring events once, and not every time they show up

Apr 27

Claude responded: Found the bug.

Diagnosed infinite loop bug preventing recurring event expansion

Found the bug. In Parsing.py, the `expand_event` while loop has no upper-bound break condition. For recurring events with no COUNT or UNTIL (the most common type — just an ongoing weekly/daily event), the loop runs forever and either hangs the app or exhausts memory before reaching the window filter. The fix is a one-liner: bail out of the loop as soon as candidates have passed the window.

Ran 4 commands, edited a file, read a file

All 11 tests still pass. The fix was a single line added at the top of the while True loop in `expand_event`:

```
python
```

```
if current_start > end_window + timedelta(days=1):
```

```
    break
```

Why this caused the bug: The loop only had two stopping conditions — COUNT or UNTIL. Most real-world recurring calendar events (e.g. "every Monday at 10am, indefinitely") have neither — they just repeat forever. Without the upper-bound break, `expand_event` would loop infinitely trying to generate every occurrence from the event's original start date to the end of time. The app would hang or crash before ever displaying anything, so recurring events effectively appeared to vanish or show only once if they happened to fail gracefully somewhere up the call stack.

With the fix, the loop stops as soon as candidates pass the display window, so a "repeats every Monday forever" event correctly generates one Event object per Monday within your 7-day view.

Parsing

PY

You said: can you show me where this is added

can you show me where this is added

Apr 27

Claude responded: The new line is at line 301, right at the top of the while True loop.

Viewed 2 files

The new line is at **line 301**, right at the top of the while True loop. Here's the before/after:

Before — the loop only knew how to stop via COUNT or UNTIL:

python

while True:

```
    current_start = event.start + (delta * i)
```

```
    if until is not None and current_start > until: # only stop #1
```

```
        break
```

```
    if count is not None and i >= count:           # only stop #2
```

```
        break
```

After — a third stop condition is added *first*, before the others:

python

while True:

```
    current_start = event.start + (delta * i)
```

```
    if current_start > end_window + timedelta(days=1): # ← NEW: stop #1
```

```
        break
```

```
    if until is not None and current_start > until:
```

```
        break
```

```
    if count is not None and i >= count:
```

```
        break
```

It's placed first so that no matter what, once the candidate dates have scrolled past your 7-day display window, the loop exits — even if the event has no COUNT or UNTIL defined.

You said: that did not fix the problem, it is still doing the same thing, i think it has to do with the recurring events taking place on mwf in the same week at the sam...

that did not fix the problem, it is still doing the same thing, i think it has to do with the recurring events taking place on mwf in the same week at the same time

Apr 28

Claude responded: Found the real bug.

Diagnosed recurring event expansion flaw in custom parser logic

Found the real bug. There are actually **two** expansion systems in this codebase:

1. `expand_event` in `Parsing.py` — your custom one, which only handles DAILY and WEEKLY with a simple timedelta, and **ignores BYDAY** entirely
2. `expand_recurring_events` in `room_availability_app.py` — uses `dateutil.rrulestr` which correctly handles BYDAY=MO,WE,FR

The problem is `parse_calendar_events` calls `parse_from_calendar` (which uses the broken #1), and `expand_recurring_events` (#2) is **never called at all**.

The fix is to change `parse_calendar_events` to use `parse_ics` for raw parsing, then feed those events through the already-correct `expand_recurring_events`:

Ran a command, edited a file

Also need to fix `refresh_calendar_data` which has the same problem:

Edited a file, ran a command, read a file

The root cause: your custom `expand_event` in `Parsing.py` only knows how to do `FREQ=DAILY` and `FREQ=WEEKLY` with a plain timedelta. A MWF event looks like `FREQ=WEEKLY;BYDAY=MO,WE,FR` in the ICS file — it has a BYDAY field that your code never reads, so it only generates one occurrence per week (on the original day) instead of three.

`expand_recurring_events` in `room_availability_app.py` was already written correctly — it passes the whole rule string to `dateutil.rrulestr`, which handles BYDAY natively. It just wasn't being called. The fix routes `parse_calendar_events` and `refresh_calendar_data` through that function instead of `parse_from_calendar`.

Room availability app

PY

You said: the file you provided now doesnt prvide the reacurring events, and it now gives me this error

the file you provided now doesnt prvide the reacurring events, and it now gives me this error

Failed to expand recurring event

040000008200E00074C5B7101A82E008000000002D8129850050DB010000000000000000
0100000007D040C0930E75E428871648BD7DE6522: unsupported property: 13 Failed to
expand recurring event

040000008200E00074C5B7101A82E00800000000289919D7FF4FDB010000000000000000
010000000719F1D1216C2BF4D8AABF7AAE8822A4F: unsupported property: 14 Failed to
expand recurring event

040000008200E00074C5B7101A82E00800000000903F9DA9E265DB010000000000000000
01000000099CAFE3D0672094A9B3BC995913E7033: unsupported property: 17 Failed to
expand recurring event

040000008200E00074C5B7101A82E00800000000A4C7E92A0050DB010000000000000000
0100000005B4565A518763F4DBE3F0308564F8FD7: unsupported property: 14 Failed to
expand recurring event

040000008200E00074C5B7101A82E008000000001AC93AE30050DB010000000000000000
010000000D95BBFC13BC1F245B9C9456CF783D1F5: unsupported property: 17 Failed to
expand recurring event

040000008200E00074C5B7101A82E0080000000083481D120150DB010000000000000000
010000000EBEB28A7E5281448B0E9D3A6E304CB00: unsupported property: 19 Failed to
expand recurring event

040000008200E00074C5B7101A82E008000000002B30A6F84215DC010000000000000000
010000000FD6342652E2BC040A713073A5BDB3CFC: unsupported property: 15 Failed to
expand recurring event

040000008200E00074C5B7101A82E0080000000038A41DB74315DC010000000000000000
01000000018C4097C5F2B0B479D90F63BD227D549: unsupported property: 16 Failed to
expand recurring event

040000008200E00074C5B7101A82E00800000000C830FC7D4315DC010000000000000000
0010000000F00EC6C512E08148833B399076F3757C: unsupported property: 18 Failed to
expand recurring event

040000008200E00074C5B7101A82E00800000000CA13743C4315DC010000000000000000
0100000007F02C7B22F10A74C82B69E32227B8142: unsupported property: 14 Failed to
expand recurring event

040000008200E00074C5B7101A82E00800000000B471D9EE4315DC010000000000000000
0100000003A4865F4F650C040ADBFBA3F646CB749: unsupported property: 18 Failed to
expand recurring event
040000008200E00074C5B7101A82E00800000000D4526D82DC20DC010000000000000000
0010000000B1C60B4C92535B49AE3D318D4E2C4FF2: unsupported property: 22 Failed to
expand recurring event
040000008200E00074C5B7101A82E008000000009F7FD0215D83DC010000000000000000
01000000085B6EF6DCEBB5843A557414CDE8CCF78: unsupported property: 14 Failed to
expand recurring event
040000008200E00074C5B7101A82E008000000007239106C5C83DC010000000000000000
010000000E4300DA85C379C42B9C738337E747B83: unsupported property: 18 Failed to
expand recurring event
040000008200E00074C5B7101A82E0080000000071C5BACE5C83DC010000000000000000
0010000000A952D8091A518B4EBDAA58F10495D9DA: unsupported property: 13 Failed to
expand recurring event
040000008200E00074C5B7101A82E00800000000CC00A87F5D83DC010000000000000000
0010000000C2F756D53EAA0644B780B49BD863EB93: unsupported property: 14 Failed to
expand recurring event
040000008200E00074C5B7101A82E00800000000BC80574E5D83DC010000000000000000
0100000009B5D3FBE23205A49AADF27E4516F9AA7: unsupported property: 17 Failed to
expand recurring event
040000008200E00074C5B7101A82E00800000000D6A6D0C70B90DC010000000000000000
0010000000C43CFD8D48A07742BFC6667A0CEB8A6F: unsupported property: 21

Apr 28

Claude responded: The error is coming from rruletostring — it's reconstructing the RRULE string from a dict, but icalendar stores values as special types (vWeekday, datetime obj...

Diagnosed RRULE serialization bug, advocated storing raw string

The error is coming from `rrule_to_string` — it's reconstructing the RRULE string from a dict, but icalendar stores values as special types (vWeekday, datetime objects, etc.) that don't serialize to the format dateutil expects. The fix is to store the raw RRULE string directly in `Parsing.py` instead of converting to a dict, then `rrule_to_string` just passes it through as-is.

Ran 3 commands, edited a file, viewed a file

Now update `expand_event` in `Parsing.py` to handle both string rrules (from real ICS files) and dict rrules (from the tests), so the tests still pass:

Ran 2 commands, viewed a file

All 11 tests pass. Now copy both fixed files to output:

Ran a command, read a file

Two files changed, both needed together:

Parsing.py — two changes:

1. `parse_ics` now stores the RRULE as a raw string (`rrule_component.to_ical().decode("utf-8")`) instead of a dict. This preserves the exact RFC 5545 format like `FREQ=WEEKLY;BYDAY=MO,WE,FR;WKST=SU` that `dateutil` needs.
2. `expand_event` now detects string rrules and routes them through `dateutil.rrulestr`, which handles `BYDAY`, `WKST`, `UNTIL` with timezones, and everything else. The old dict-based path is kept intact so all 11 tests still pass.

room_availability_app.py — the fix from before (routing through `expand_recurring_events` instead of `parse_from_calendar`) is still in place. Now that `Parsing.py` passes a proper string, `rrule_to_string` has nothing tricky to convert — it just returns the string directly via its existing `if isinstance(rrule_dict, str): return rrule_dict` check.

Parsing

PY

Room availability app

PY

You said: does this still work with the parsing test

does this still work with the parsing test

Apr 28

Claude responded: Yes — I ran the full test suite as part of the fix and all 11 passed:

Recalled prior test execution results

Yes — I ran the full test suite as part of the fix and all 11 passed:

`parser_test.py::test_non_recurring_event_returns_itself` PASSED

`parser_test.py::test_non_recurring_event_outside_window` PASSED

```
parser_test.py::test_daily_recurrence_basic PASSED
parser_test.py::test_recurrence_clipped_to_window PASSED
parser_test.py::test_exdate_removes_occurrence PASSED
parser_test.py::test_weekly_recurrence PASSED
parser_test.py::test_rrule_until PASSED
parser_test.py::test_duration_preserved PASSED
parser_test.py::test_all_day_event_recurrence PASSED
parser_test.py::test_event_with_no_end PASSED
parser_test.py::test_uid_preserved PASSED
```

11 passed in 0.78s

The tests still work because they construct Event objects directly with dict rrules (via `make_event`), and `expand_event` checks `isinstance(event.rrule, str)` — if it's a dict, it falls through to the original dict-based logic unchanged. The new string path only activates for events that came through `parse_ics`, which is what real ICS files produce.

You said: how do i run the test

how do i run the test

Apr 28

Claude responded: Since you're using a virtual environment in your dev container, run:

Since you're using a virtual environment in your dev container, run:

```
bash
```

```
pytest parser_test.py -v
```

or if pytest isn't on your PATH:

```
bash
```

```
python -m pytest parser_test.py -v
```

The -v flag gives you the verbose output showing each test name and pass/fail individually. Based on your TDD log, it looks like you've been running it from the /workspaces/Software-Engineering-Project directory with the .venv active, so:

```
bash
```

```
cd /workspaces/Software-Engineering-Project
```

```
source .venv/bin/activate
```

```
pytest parser_test.py -v
```

You said: should i include the pycache on the github?

should i include the pycache on the github?

Apr 28

Claude responded: No — you should exclude it.

No — you should exclude it. Add a .gitignore file to your project root with:

```
__pycache__/
```

```
*.pyc
```

If you've already committed __pycache__ to your repo, you'll need to remove it from tracking:

```
bash
```

```
git rm -r --cached __pycache__
```

```
git commit -m "Remove pycache from tracking"
```

__pycache__ is auto-generated by Python every time you run your code, is machine-specific, and adds no value to a repo. Same goes for the .venv folder if that's not already in your .gitignore.